

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**

Controle de versão com Git e GitHub

Fundamentos de Git

Aula 2

Código da aula: [DADOS]ANO1C1B3S20A2

Controle de versão
com Git e GitHub

Mapa da Unidade 8 Componente 1

Branches no Git

semana
25

Branches e Merging

semana
27

semana
20

Você está aqui!

Fundamentos de Git

semana
23

Fluxo de trabalho
básico com o Git

semana
29

GitHub



**Controle de versão
com Git e GitHub**

Mapa da Unidade 8 Componente 1

Você está aqui!

Fundamentos de Git

20

Aula 2

Código da aula: [DADOS]ANO1C1B3S20A2



Objetivos da aula

- Vamos nos aprofundar no conceito de “O que é um *commit*” e entender algumas preocupações importantes com *commits*.



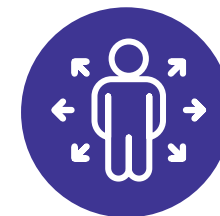
Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática e/ou à internet.



Duração da aula

50 minutos



Competências técnicas

- Usar ferramentas de desenvolvimento de software, como Git e GitHub.



Competências socioemocionais

- Trabalhar em equipes multifuncionais, apoiando colegas, gestores e clientes para alcançar objetivos comuns.

Construindo
o **conceito**

Commits

- ▶ Vimos, no começo da aula passada, a estrutura de um repositório Git, mas nunca nos aprofundamos muito em como ele funciona.
- ▶ Falando sobre *commits* em particular: o que de fato um *commit* representa?

Desvendando o processo de *commit* no Git

- ▶ Um *commit* no Git é um *snapshot* do seu projeto, capturando o estado atual de todos os arquivos rastreados.
- ▶ Quando você faz um *commit*, o Git calcula um *hash* SHA-1 (um cálculo matemático complexo) embasado no conteúdo dos arquivos, criando um objeto *commit* que inclui esse *hash*, um ponteiro para o *commit* pai, autor e mensagem de *commit*.

Ciclo de vida de um *commit*

- ▶ O ciclo de vida de um *commit* começa com a *staging area*, em que você prepara (*stage*) as mudanças (como pelo uso do comando *git add*, por exemplo).
- ▶ Ao executar o *commit*, o Git cria um novo objeto *commit* no repositório local (como é feito no comando *git commit*).
- ▶ Este *commit* só é compartilhado com outros desenvolvedores quando você faz *git push* para um repositório remoto.
- ▶ Discutimos, na aula passada, sobre o que acontece quando fazemos o *commit* de algo como os dados de ambiente do projeto (dado supersensível). Vamos responder a essa pergunta agora!

Opções do Git

- ▶ Ao iniciar um repositório Git em uma pasta, são criadas duas entidades: a pasta `.git` (que é uma pasta oculta; logo, só será visualizada se as pastas ocultas estiverem visíveis) e o arquivo `.gitignore`. Essas estruturas são importantes para definirmos configurações/opções no Git.
- ▶ Nesta aula em particular, falaremos sobre o arquivo `.gitignore` e sobre os *hooks* do Git.

Entendendo o .gitignore

- ▶ O arquivo .gitignore permite especificar quais arquivos ou diretórios o Git deve ignorar e não rastrear. Isso é crucial para evitar que arquivos temporários ou sensíveis sejam enviados a um repositório remoto.
- ▶ Você pode configurar esse arquivo para excluir padrões específicos de nomes de arquivos, como logs, arquivos binários, e definir extensões de arquivo que não devem ser rastreadas ou pastas de configuração pessoal.

Como configurar um **.gitignore**

- ▶ Para configurar um .gitignore, crie ou edite o arquivo na raiz do seu projeto. Adicione padrões de nome de arquivo, um por linha, para excluir do rastreamento.
- ▶ Para excluir uma pasta, utilize o seu nome seguido de uma barra enquanto temp/ignora toda a pasta temp.
- ▶ Para excluir uma extensão, escreva *.extensão. Por exemplo, *.log ignora todos os arquivos com a extensão .log.
- ▶ Para excluir um arquivo específico, escreva seu caminho (incluindo as pastas, se ele não estiver na raiz).

O que são *Git Hooks*?

- ▶ *Git Hooks* são scripts que o Git executa antes ou depois de eventos, como *commits*, *push* e *receive*.
- ▶ Eles são usados para automatizar tarefas, como testes de código, verificações de estilo ou qualquer outra operação que você deseje executar automaticamente em resposta a eventos do Git.

Exemplos práticos de *Git Hooks*

- ▶ Um exemplo comum é o *pre-commit hook*, que pode ser configurado para executar um linter (script de formatação do código) ou testes automáticos antes de cada *commit*.
- ▶ Se os testes falharem, o *commit* é abortado, garantindo que apenas código que passa nos testes seja “commitado”.



Vamos
fazer um
quiz

O que o arquivo *.gitignore* faz?

Lista arquivos para serem
incluídos em cada *commit*.

Especifica arquivos e diretórios
que o Git deve ignorar.

Configura *hooks* do Git.

Armazena informações de
configuração do usuário.



Vamos
fazer um
quiz

Qual a função de um *pre-commit hook* no Git?

Enviar um e-mail para a equipe após cada *commit*.

Fazer *push* do *commit* para o repositório remoto automaticamente.

Executar testes ou verificações antes que o *commit* seja efetivado.

Compilar o projeto após cada *commit*.



Vamos
fazer um
quiz

Como o Git determina o que incluir em um novo *commit*?

Automaticamente, inclui todos os arquivos modificados.

Inclui apenas arquivos especificados no *.gitignore*.

Inclui mudanças que estão na *staging area*.

Inclui todos os arquivos do diretório atual.



© Getty Images

O que nós
**aprendemos
hoje?**

Então, ficamos assim...

- 1** Descrevemos detalhadamente como um *commit* é processado no Git, desde a adição de mudanças à *staging area* até a criação do objeto *commit* no repositório local.
- 2** Exploramos como o arquivo `.gitignore` é essencial para especificar quais arquivos e diretórios o Git deve ignorar, evitando o rastreamento de arquivos desnecessários ou sensíveis, e mantendo o repositório limpo e eficiente.
- 3** Introduzimos o conceito de *Git Hooks*, destacando como esses scripts podem ser utilizados para automatizar tarefas, como testes e verificações antes de *commits*, melhorando a qualidade do código e a eficiência do processo de desenvolvimento.

Saiba mais

Interessou-se pelos *Git Hooks*? Eles são uma ferramenta muito interessante para dominar e ter um currículo mais completo.

Veja mais no artigo que está no link abaixo!

GONÇALVES, A. *Como utilizar git hooks?* Hostinger, 19 jan. 2023. Disponível em:

<https://www.hostinger.com.br/tutoriais/como-usar-git-hooks/>. Acesso em: 30 maio 2024.

Referências da aula

BELL, P. ; BEER, B. *Introdução ao GitHub*: um guia que não é técnico. São Paulo: Novatec, 2014.

CHACON, S; STRAUB, B. *Pro Git*. Apress, 2024. Disponível em: <https://git-scm.com/book/pt-br/v2/>.

Acesso em: 30 maio 2024.

GITHUB DOCS. *Documentação de Introdução ao GitHub*, [s.d.]. Disponível em: <https://docs.github.com/pt/get-started/>. Acesso em: 30 maio 2024.

Identidade visual: imagens © Getty Images.

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**